CampaignWe Data Pipeline

Overview

This pipeline processes click data extracted from Azure Application Insights for an internal employee engagement campaign page. It ingests KQL exports (`.xlsx` or `.csv`), enriches them with organisational data, computes derived metrics, and exports Parquet files for reporting.

> **Terminology**: Every record in this pipeline represents a **click** — a user clicking a link, button, or story on the page. The source Application Insights event type is `click_event`. Database tables and columns use the name `events` (inherited from App Insights conventions), but these are always click events.

```
Application Insights (KQL)
        |
        v
  input/*.xlsx / *.csv        <-- you drop files here
        |
        v
  process_campaignwe.py        <-- delta detection + upsert + enrichment
        |
        +---> data/campaignwe.db                 (DuckDB database)
        +---> output/events_raw.parquet          (internal: all events
with raw identifiers)
        +---> output/events_anonymized.parquet   (primary: anonymised,
visitor_* org fields)
        +---> output/story_metadata.parquet      (story lookup: text,
keys, author info)
```

Input Files

Source

Export click events from Application Insights using the KQL query in `campaignwe_query.kql`. The query targets `customEvents` where `name == "click_event"`.

File Naming Convention

```
campaign_export_YYYY_MM_DD.xlsx
campaign_export_YYYY_MM_DD.csv
```

The `_YYYY_MM_DD` date suffix is mandatory for correct ordering. Files without a date suffix fall back to filesystem modification time.

Placement

Drop files into the `input/` folder. The script scans this folder automatically.

Format Recommendations

Format	Timestamp Precision	GPN Handling	Recommendation
CSV	Microsecond (full)	String (safe)	Preferred
XLSX	Second only (truncated)	May lose leading zeros	Use if CSV unavailable

CSV preserves the full `dd/MM/yyyy HH:mm:ss.fffffff` timestamp from App Insights. Excel truncates to whole seconds, which weakens the composite primary key's uniqueness.

Expected Columns

Column	Source	Notes
`timestamp [UTC]`	App Insights	Renamed to `timestamp` during load
`name`	App Insights	Event type, typically `click_event`
`user_Id`	App Insights	Renamed to `user_id`
`session_Id`	App Insights	Renamed to `session_id`
`client_CountryOrRegion`	App Insights	Geographic info
`CP_GPN` / `CP_gpn`	CustomProps	Global Personnel Number (8 digits)
`CP_Email`	CustomProps	User email
`CP_Link_label`	CustomProps	Parsed for story_id and action_type

Running the Script

Prerequisites

```
pip install duckdb pandas openpyxl
```

Usage

```
# Delta mode (default) -- process only new or changed files
python process_campaignwe.py

# Force-process a specific file (bypasses delta check)
python process_campaignwe.py input/campaign_export_2026_02_25.xlsx

# Full refresh -- delete database and reprocess all files from scratch
python process_campaignwe.py --full-refresh
```

Typical Workflow

1. Export data from App Insights (daily or weekly)
2. Save the `.xlsx` or `.csv` file to `input/`
3. Run `python process_campaignwe.py`
4. Use the output parquet files for reporting

Delta Processing

The script tracks which files have already been processed using a `processed_files` manifest table inside the DuckDB database.

How It Works

On each run (without `--full-refresh` or a specific file argument):

1. **Scan** `input/` for all `.xlsx`, `.xls`, `.csv` files
2. **Hash** each file's contents (SHA-256)
3. **Compare** against the `processed_files` table in the database:
   - **New filename** -- file is processed
   - **Same filename, same hash** -- file is skipped (already processed)
   - **Same filename, different hash** -- file is re-processed (contents changed)
4. **Record** successfully processed files in the manifest

Manifest Table Schema

```
CREATE TABLE processed_files (
    filename     TEXT PRIMARY KEY,    -- e.g.
"campaign_export_2026_02_20.xlsx"
    file_hash    TEXT,                -- SHA-256 of file contents
    row_count    INTEGER,             -- rows loaded from this file
    processed_at TIMESTAMP,           -- when processing occurred
    date_suffix  DATE                 -- extracted YYYY_MM_DD from filename
);
```

Behaviour by Scenario

Scenario	What Happens
First run, 3 files in `input/`	All 3 processed oldest-first, all recorded in manifest
Second run, no new files	"All files already processed. Nothing new to do."
New file added to `input/`	Only the new file is processed
File replaced (same name, new content)	Hash mismatch detected, file re-processed
`--full-refresh`	Database deleted (including manifest), all files reprocessed

Scenario	What Happens
Explicit file argument	File is force-processed regardless of manifest state

Upsert Logic (Overlap Handling)

Weekly or daily exports from App Insights may contain overlapping date ranges. The script uses a **delete-then-insert** upsert pattern to prevent double-counting.

Primary Key

```
(timestamp, user_id, session_id, name)
```

Mechanism

```sql
-- Step 1: Delete existing rows that match incoming rows on the composite
key
DELETE FROM events_raw
WHERE EXISTS (
    SELECT 1 FROM temp_import t
    WHERE events_raw.timestamp = t.timestamp
      AND events_raw.user_id = t.user_id
      AND events_raw.session_id = t.session_id
      AND events_raw.name = t.name
);

-- Step 2: Insert all rows from the new file
INSERT INTO events_raw SELECT * FROM temp_import;
```

Example

If file A covers Jan 1-14 and file B covers Jan 10-21:

- Events from Jan 10-14 appear in both files
- When file B is loaded, matching rows from Jan 10-14 are deleted from `events_raw` first
- Then all of file B's rows (Jan 10-21) are inserted
- Result: no duplicates, file B's data takes precedence for the overlap period

Precision Warning

The composite key relies on timestamp uniqueness. If two identical events from the same user/session occur in the same second (possible when Excel truncates microseconds), they will be treated as one event. **Export as CSV to preserve microsecond precision.**

Processing Pipeline

After file loading and upsert, the script runs these stages:

1. Organisational Data Enrichment

The pipeline enriches click events with organisational fields (division, unit, area, sector, region, country, etc.) using an internal data source. This produces `visitor_*` columns in the anonymised output.

2. Calculated Columns

Column	Description
`person_hash`	Anonymised user identifier (SHA-256 hash)
`story_id`	Extracted from `CP_Link_label` via leading digits pattern
`action_type`	Classified from `CP_Link_label` (see mapping below)
`timestamp_cet`	UTC converted to Europe/Berlin timezone
`session_date`	CET-based date (for daily bucketing)
`session_key`	`YYYY-MM-DD_user_id_session_id` (unique session identifier)
`event_hour`	Hour in CET (0-23)
`event_weekday`	Day name (Monday, Tuesday, ...)
`event_order`	Sequence number within session
`prev_event` / `prev_timestamp`	Previous event in session (for flow analysis)
`ms_since_prev_event`	Milliseconds since previous event
`time_since_prev_bucket`	Categorised interval (< 0.5s, 0.5-1s, 1-2s, ..., > 60s)
`visitor_division` through `visitor_function`	Organisational hierarchy fields
`visitor_region`, `visitor_country`	Geographic fields

Action Type Classification

The `action_type` column is derived from the `CP_Link_label` text using pattern matching (case-insensitive):

`CP_Link_label` Pattern	`action_type`	Description
`%Share your story%`	**Open Form**	User opened the story submission form
`%Submit%`	**Submit**	User submitted a story
`%Send Invite%`	**Send Invite**	User sent an invite to a colleague
`%Invite your colleagues%`	**Open Invite**	User opened the invite form

`CP_Link_label` Pattern	`action_type`	Description
`%Cancel%`	**Cancel**	User cancelled/closed a form
`%Read%`	**Read**	User opened/expanded a story
`%like%`	**Like**	User liked content
Anything else	**Other**	Unclassified click (excluded from reporting)

Other groups clicks that add no analytical value: closing a story after reading it (`close`), editing form fields (`edit`), browsing/pagination (`See more stories`, pure digit clicks), and events with no label (`NULL`). The processing summary shows all distinct "Other" labels with counts for review.

The `story_id` is extracted from the leading digits in `CP_Link_label` — e.g., `"15Read full story"` yields `story_id = 15`.

Export Filter

The anonymized export (`events_anonymized.parquet`) applies these rules:

Action Type	Included?	Condition
Read, Like	Yes	Only if `story_id` matches a known story in `story_metadata.parquet`
Open Form, Submit, Cancel	Yes	Always (story creation funnel)
Send Invite, Open Invite	Yes	Always (invite funnel)
Other	No	Always excluded

This ensures clean funnel analysis while filtering out noise. The raw database retains all events for diagnostics.

Views per Visitor

Used in engagement tables:

```
views_per_visitor = COUNT(action_type = 'Read') / COUNT(DISTINCT
person_hash)
```

This is the average number of story-open clicks per unique visitor within a given grouping (division, region, etc.).

3. Parquet Export

File	Contents	Grain

File	Contents	Grain
`events_raw.parquet`	All events with raw identifiers and org columns (internal use only)	One row per event
`events_anonymized.parquet`	Primary export: filtered to known stories + funnel actions, identifiers hashed/dropped, `visitor_*` org fields	One row per event
`story_metadata.parquet`	Story lookup with `story_text`, `story_title` (when available), author info (email, division, department, job title, country, business sector, area, unit), keys	One row per story

Output Database

The DuckDB database at `data/campaignwe.db` contains:

Table	Description
`events_raw`	Raw imported data (pre-enrichment) — dropped after enrichment
`events`	Final enriched table with all calculated columns
`story_titles`	Story metadata (loaded each run from `story_metadata.parquet`, dropped after join)
`processed_files`	File processing manifest for delta tracking

Troubleshooting

"All files already processed"

The manifest shows all files have matching hashes. Either:

- Drop a new file into `input/` and re-run
- Use `--full-refresh` to reprocess everything
- Pass a specific file path to force-process it

Timestamp precision warning

```
WARNING: Column 'timestamp' has no microsecond precision!
```

This means the input file (likely `.xlsx`) has truncated timestamps. Export from App Insights as CSV instead.

Unmatched Visitors

Visitors appearing in events but without organisational data are shown in the summary. Common causes:

- Identifier format mismatch
- Employee not in the organisational data timeframe
- External or contractor accounts